
Post-Quantum Blockchain Architecture

Using Lattice-Based Cryptography for Critical Infrastructure Security

A 12-Month Research & Development Execution Plan

Monthly Deliverables, Quarterly Milestones, and Tooling Strategy

Research Domains Covered

Hybrid Signature Migration Protocols · Lattice Signature Compression & Aggregation
Smart Contract VM Precompiles for Dilithium · Forward-Secure Validator Key Management
QROM Security Proofs for Consensus Protocols

Prepared for	Harshavarthan S – M.E. CSE, SSN College of Engineering, Chennai
Project Duration	12 Months (Month 1 – Month 12)
Document Type	Research & Development Execution Roadmap
Version	1.0

Contents

1	Project Overview & Rationale	2
1.1	Five Core Problems Addressed	2
1.2	Plan Structure	2
2	Quarterly Milestone Summary	2
3	Month-by-Month Deliverables	4
3.1	Quarter 1 – Foundations & Threat Modelling	4
3.2	Quarter 2 – Hybrid Migration & Signature Compression	4
3.3	Quarter 3 – VM Precompile & Validator Key Security	6
3.4	Quarter 4 – QROM Proofs, Integration & Dissemination	6
4	Timeline at a Glance	8
5	Tools, Technologies & Their Purpose	8
5.1	Cryptographic Libraries	8
5.2	Blockchain & Smart-Contract Tooling	9
5.3	Formal Methods & Proof Tooling	9
5.4	Benchmarking, Simulation & Data Analysis	10
5.5	Documentation & Collaboration	10
6	Key Dependencies & Risk Notes	11
7	Final Deliverable Set (End of Month 12)	11

1 Project Overview & Rationale

Every operational blockchain in production today – Bitcoin, Ethereum, Hyperledger – depends on ECDSA for signatures and SHA-256/Keccak for hashing. Shor’s algorithm breaks ECDSA in polynomial time on a sufficiently large quantum computer, while Grover’s algorithm weakens (but does not break) SHA-256. With NIST’s 2024 finalization of CRYSTALS-Dilithium (signatures) and CRYSTALS-Kyber (key encapsulation), the cryptographic building blocks for a post-quantum blockchain now exist – but the *systems architecture* to deploy them at scale, without breaking throughput, smart-contract compatibility, or validator liveness, does not.

This is especially urgent for **critical infrastructure** – power grids, water treatment, and railway signaling systems – which increasingly rely on distributed ledgers for command authentication and audit trails, and which operate on 30-year horizons. A system commissioned today will very plausibly face quantum-capable adversaries before it is decommissioned.

1.1 Five Core Problems Addressed

- **P1 – Live Migration:** Hybrid ECDSA/Dilithium transition protocol across live nodes without a disruptive hard fork.
- **P2 – Signature Bloat:** Dilithium signatures are $\sim 2.4\text{KB}$ vs. ECDSA’s 64–72 bytes (a $33\times$ increase); block structures and verification must be redesigned to absorb this.
- **P3 – VM Compatibility:** No major smart-contract VM exposes a lattice-verification precompile; naive Solidity verification is gas-prohibitive.
- **P4 – Validator Key Management:** Forward-secure lattice signatures and hash-based schemes (XMSS/SPHINCS+) to prevent retroactive forgery of consensus history.
- **P5 – QROM Security Proofs:** Formal security reductions in the Quantum Random Oracle Model, which is strictly stronger than the classical random oracle model most blockchain proofs rely on.

1.2 Plan Structure

This roadmap decomposes the work into **12 monthly sprints**, grouped into **4 quarterly milestones**, each targeting one or more of the five core problems while building toward a single integrated, formally-analyzed, benchmarked prototype and a publication-ready thesis/paper package.

2 Quarterly Milestone Summary

Q1 (Months 1–3) – Foundations & Threat Modelling

Literature synthesis, formal threat model for critical-infrastructure DLT, and a working Dilithium/Kyber signing-and-verification sandbox. **Exit criterion:** approved research proposal + reproducible crypto sandbox benchmark report.

Q2 (Months 4–6) – Hybrid Migration & Signature Compression (P1, P2)

Design and simulate the hybrid ECDSA/Dilithium transition protocol; prototype signature aggregation/compression and redesigned block structure. **Exit criterion:** simulated testnet demonstrating hybrid consensus with measured throughput under Dilithium load.

Q3 (Months 7–9) – VM Precompile & Validator Key Security (P3, P4)

Implement a lattice-verification precompile / ZK-wrapper prototype for an EVM-like VM; implement and benchmark forward-secure validator key schemes (XMSS/SPHINCS+). **Exit criterion:** working precompile gas-cost benchmark + validator key-rotation demo resistant to retroactive forgery.

Q4 (Months 10–12) – QROM Proofs, Integration & Dissemination (P5)

Formal/semi-formal QROM security reduction for the integrated architecture; full system integration; performance evaluation; manuscript and thesis chapter writing; submission to a target venue. **Exit criterion:** integrated prototype + security proof sketch + submitted paper/thesis draft.

3 Month-by-Month Deliverables

3.1 Quarter 1 – Foundations & Threat Modelling

Month	Focus & Activities	Deliverables
Month 1	Systematic literature review: PQC blockchain proposals, NIST PQC standards (Dilithium, Kyber, SPHINCS+), existing hybrid-signature literature, QROM proofs in distributed systems. Set up development environment and version control.	Annotated bibliography (≥ 40 sources); gap-analysis note identifying unsolved sub-problems within P1–P5; project GitHub repo initialized with CI skeleton.
Month 2	Formal threat model for critical-infrastructure DLT (power grid / water treatment / rail signaling command authentication). Define adversary capabilities (classical vs. quantum, harvest-now-decrypt-later). Stand up a local Dilithium/Kyber signing sandbox using liboqs / PQCclean.	Threat-model document (STRIDE-style, quantum-adversary variant); working crypto sandbox producing/verifying Dilithium signatures and Kyber key exchanges; baseline micro-benchmarks (sign/verify latency, key sizes).
Month 3	Baseline blockchain testbed selection (e.g., a permissioned Hyperledger Fabric or a minimal custom PoS chain) with vanilla ECDSA signatures. Benchmark current throughput/latency to serve as the "before" baseline for later comparison. Draft formal research proposal and Q1 milestone review.	Baseline blockchain testbed running end-to-end with ECDSA; baseline performance report (TPS, block time, signature verification cost); Milestone 1 report consolidating literature, threat model, and baseline benchmarks.

3.2 Quarter 2 – Hybrid Migration & Signature Compression

Month	Focus & Activities	Deliverables
Month 4	Design the hybrid ECDSA/Dilithium transition protocol: dual-signature transaction format, node capability negotiation, and finality-preserving fork-choice rule during the migration window.	Hybrid protocol specification document (state diagrams, message formats); threat analysis of the "mixed cryptography" attack window.

Month	Focus & Activities	Deliverables
Month 5	Implement hybrid signature validation into the baseline testbed (accept both ECDSA and Dilithium transactions concurrently). Begin work on signature compression: evaluate Dilithium aggregation schemes and batch verification.	Working hybrid-signature testbed (both schemes co-existing on one chain); prototype batch-verification module with measured verification-time savings.
Month 6	Redesign block structure to accommodate larger Dilithium payloads (variable-size signature fields, Merkleized signature commitments to keep header size constant). Run full migration simulation on testnet under realistic transaction load. Quarter 2 milestone review.	Redesigned block-format specification + implementation; migration simulation report comparing pre/post-migration throughput, block size, and propagation latency; Milestone 2 report .

3.3 Quarter 3 – VM Precompile & Validator Key Security

Month	Focus & Activities	Deliverables
Month 7	Study EVM precompile architecture and gas-cost model. Design a lattice-verification precompile specification for Dilithium; alternatively, prototype a ZK-SNARK wrapper that compresses a Dilithium verification into a succinct proof checkable by an existing precompile.	Precompile specification document; feasibility comparison table (native precompile vs. ZK-wrapper vs. alternate execution environment) with estimated gas costs.
Month 8	Implement chosen approach (precompile in a modified EVM client, or ZK-circuit for Dilithium verification using a framework such as Circom/Halo2). Benchmark gas cost against a naive on-chain Solidity verifier.	Working precompile / ZK-verifier prototype; gas-benchmark report showing cost reduction relative to naive implementation; smart-contract test suite.
Month 9	Implement forward-secure validator key management: XMSS and SPHINCS+ key-rotation scheme for PoS validators. Demonstrate resistance to retroactive attestation forgery after simulated key compromise. Quarter 3 milestone review.	Validator key-management module (key generation, rotation, revocation); forgery-resistance demonstration report; Milestone 3 report covering precompile and validator-key results.

3.4 Quarter 4 – QROM Proofs, Integration & Dissemination

Month	Focus & Activities	Deliverables
Month 10	Develop the formal/semi-formal security argument for the integrated architecture in the Quantum Random Oracle Model, adapting existing QROM reduction techniques for Dilithium/Fiat-Shamir-with-aborts to the consensus-layer setting.	QROM security-reduction draft (proof sketch with clearly stated assumptions and reduction losses); internal review with faculty guide/domain expert.

Month	Focus & Activities	Deliverables
Month 11	Full system integration: hybrid migration layer + compressed signatures + VM pre-compile + forward-secure validator keys running together on one testbed. End-to-end performance evaluation against the Month-3 ECDSA baseline.	Integrated prototype (single deployable testbed); comprehensive evaluation report (throughput, latency, gas cost, signature overhead, security margin) with comparison charts.
Month 12	Consolidate results into a thesis chapter / conference paper draft (target venue: IEEE S&P, ACM CCS workshop, or a blockchain-focused venue). Prepare reproducibility package (code, datasets, benchmark scripts) and final presentation. Quarter 4 milestone review.	Camera-ready-style paper/thesis draft; open-source reproducibility repository; final presentation deck; Milestone 4 report – Final Project Report.

4 Timeline at a Glance

Workstream	M1	M2	M3	M4	M5	M6	M7	M8	M9	M10	M11	M12
Literature & Threat Model	•	•	•									
P1: Hybrid Migration			•	•	•	•						
P2: Signature Compression					•	•						
P3: VM Precompile							•	•				
P4: Validator Key Mgmt.									•			
P5: QROM Proofs										•		
Integration & Evaluation											•	
Writing & Dissemination										•	•	•

Table 5: *

Shaded cells indicate the active month(s) for each workstream. Colors correspond to Q1 (blue), Q2 (green), Q3 (amber), Q4 (purple).

5 Tools, Technologies & Their Purpose

5.1 Cryptographic Libraries

Tool	Used In	Purpose
liboqs (Open Quantum Safe)	Months 2, 5, 9	Reference C library providing Dilithium, Kyber, SPHINCS+, XMSS implementations; used to build the signing/verification sandbox and validator key modules.
PQClean	Months 2, 8	Clean, auditable reference implementations of PQC algorithms; used to cross-validate liboqs outputs and as a base for the precompile implementation.
liboqs-python / oqs-provider (OpenSSL)	Months 2, 4	Python and OpenSSL bindings for rapid prototyping of hybrid signature handshakes.

5.2 Blockchain & Smart-Contract Tooling

Tool	Used In	Purpose
Hyperledger Fabric / go-ethereum (geth, modified)	Months 3–6, 11	Baseline permissioned/PoS ledger for the ECDSA baseline and the hybrid-migration testbed; source base for the modified EVM client hosting the new precompile.
Solidity + Hardhat	Months 7–8	Writing and testing the naive on-chain Dilithium verifier used as the gas-cost baseline, and the smart-contract test suite for the precompile.
Circom / Halo2	Month 8	Building the ZK-SNARK circuit that wraps Dilithium verification into a succinct, cheaply-checkable proof if the ZK-wrapper approach is selected over a native precompile.
Geth EVM tracer / evmone	Months 7–8	Precise gas-cost profiling of the precompile vs. naive verifier.

5.3 Formal Methods & Proof Tooling

Tool	Used In	Purpose
Tamarin Prover / ProVerif	Month 2, 10	Symbolic verification of the hybrid-migration protocol's authentication and finality properties under an active-adversary model.
Pen-and-paper QROM reduction + CryptoVerif (optional mechanization)	Month 10	Constructing and sanity-checking the QROM security reduction for the integrated signature/consensus scheme.

5.4 Benchmarking, Simulation & Data Analysis

Tool	Used In	Purpose
Docker / Docker Compose	All months	Reproducible multi-node testbed deployment for every experiment (baseline, hybrid, integrated).
Mininet	Months 3, 6, 11	Network-layer simulation of validator nodes for realistic latency/propagation testing across the migration and integration experiments.
Prometheus + Grafana	Months 3, 6, 9, 11	Real-time metrics collection (TPS, block time, CPU, signature verify time) and dashboarding during benchmark runs.
Python (NumPy, Pandas, Matplotlib)	All months	Data processing and chart generation for every benchmark and evaluation report.
JMeter / custom load generator	Months 6, 11	Synthetic transaction-load generation to stress-test throughput under Dilithium-sized signatures.

5.5 Documentation & Collaboration

Tool	Used In	Purpose
GitHub (+ GitHub Actions CI)	All months	Version control for all code, automated test/build pipelines, and issue tracking against the monthly deliverables.
Overleaf / LaTeX	Months 1, 10–12	Drafting the research proposal, QROM proof write-up, thesis chapter, and final paper.
Zotero	Month 1, ongoing	Reference management for the literature review and citation tracking through the paper draft.
Notion / Obsidian	All months	Running lab notebook for weekly progress notes, decisions log, and milestone tracking.

6 Key Dependencies & Risk Notes

- **P2 depends on P1:** signature-compression work (Month 5–6) assumes the hybrid transaction format from Month 4 is stable; a slip in Month 4 pushes the whole Q2 exit criterion.
- **P3 is the highest engineering-risk workstream:** if the native-precompile route proves infeasible within Month 7’s feasibility study, the ZK-wrapper fallback must be triggered no later than the start of Month 8 to protect the Q3 deadline.
- **P5 (QROM proof) is the highest theoretical-risk workstream:** Month 10 is scoped for a proof *sketch*, not a fully mechanized proof; if faculty/expert review in Month 10 flags a gap, Month 11 buffer time should absorb the revision before integration reporting.
- **Hardware dependency:** benchmarking (Months 3, 6, 9, 11) assumes access to a multi-node cluster or cloud VMs (e.g., AWS/Azure) capable of running 4–8 validator nodes concurrently; this should be provisioned by end of Month 2.

7 Final Deliverable Set (End of Month 12)

- Integrated post-quantum blockchain prototype (hybrid migration + compressed signatures + VM precompile + forward-secure validator keys).
- Formal threat model and QROM security-reduction sketch.
- Comprehensive benchmark suite and evaluation report (baseline vs. final architecture).
- Open-source reproducibility repository (code, configs, benchmark scripts, datasets).
- Conference paper / thesis chapter draft ready for submission.